

Daffodil International University
DEPARTMENT OF INFORMATION & COMMUNICATION ENGINEERING
Dhaka, BANGLADESH.
COURSE NO.: ICE 3242
COURSE TITLE: DIGITAL SIGNAL PROCESSING Lab

Experiment No. 02

Name of Experiment: Basic Operations on Discrete-Time Signals

Objectives:

The objectives of this experiment are:

- To represent a discrete-time signal in MATLAB as a pair of vectors: the time index n and the sample values x .
- To perform the following operations on a discrete-time signal using MATLAB 7.0:
 - Amplitude scaling
 - Time shifting (delay and advance)
 - Time reversal (folding)
 - Time scaling (compression and expansion)
 - Combination of operations, e.g. $x(-n + 2)$
- To compute the correct time index of every transformed signal and verify it against a hand calculation.

THEORY:

In Experiment 01 a discrete-time signal $x(n)$ was generated as a sequence of numbers defined at integer values of the time index n . An operation on a signal changes either the **values** of the samples (dependent variable) or the **positions** of the samples along the n -axis (independent variable). In MATLAB a signal must therefore always be stored with two vectors of equal length: n (time indices) and x (sample values). The test signal used in all programs of this experiment is

$$x(n) = \{1, 2, \underset{\uparrow}{3}, 4, 5, 6\}, \quad -2 \leq n \leq 3$$

where the arrow marks the sample at $n = 0$. Its sample $x(0) = 3$ is used as a reference point to check every operation.

MATLAB indexing: MATLAB array positions always start at 1. $x(1)$ returns the *first stored sample*, which here is the value at $n = -2$, not at $n = 0$. MATLAB does not know where $n = 0$ lies; that information is carried only by the vector n . A time index is converted to an array position by
`position = n - min(n) + 1.`

1. Amplitude Scaling

Amplitude scaling multiplies every sample by a constant A . Only the sample values change; the time indices remain the same. If $|A| > 1$ the signal is amplified, if $|A| < 1$ it is attenuated, and if $A < 0$ every sample is also inverted in sign. In MATLAB this is the single statement $y = A * x$, and the index vector n is reused without change.

$$y(n) = A x(n)$$

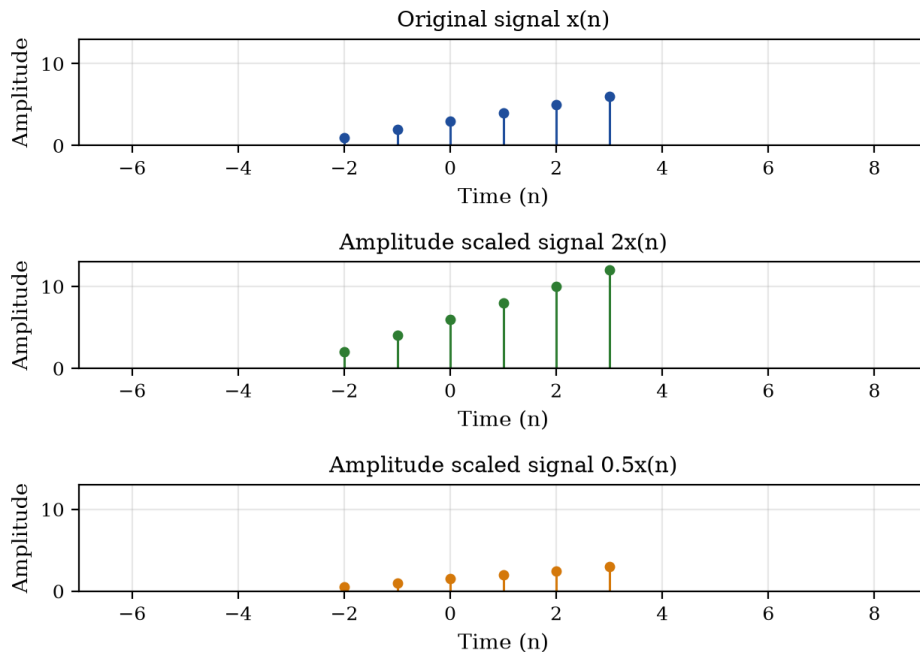


Fig. 2.1 Amplitude scaling of $x(n)$ by $A = 2$ and $A = 0.5$

2. Time Shifting

Time shifting moves the whole sequence along the n -axis by an integer number of samples k ($k > 0$). In a **delay**, $y(n) = x(n - k)$, every sample moves k positions to the *right*; in an **advance**, $y(n) = x(n + k)$, every sample moves k positions to the *left*.

$$y(n) = x(n - k) \quad (\text{delay}), \quad y(n) = x(n + k) \quad (\text{advance})$$

A simple check avoids confusion: in $x(n - k)$ the argument becomes zero when $n = k$, so the old origin $x(0)$ now appears at $n = k$. The values are not modified; only the index vector changes. In MATLAB a delay by k is therefore written $n + k$ and an advance by k is written $n - k$ — the sign is opposite to the sign inside the brackets, which is correct. A delay is physically realisable (past samples can be stored), whereas an advance needs future samples and is possible only for stored data.

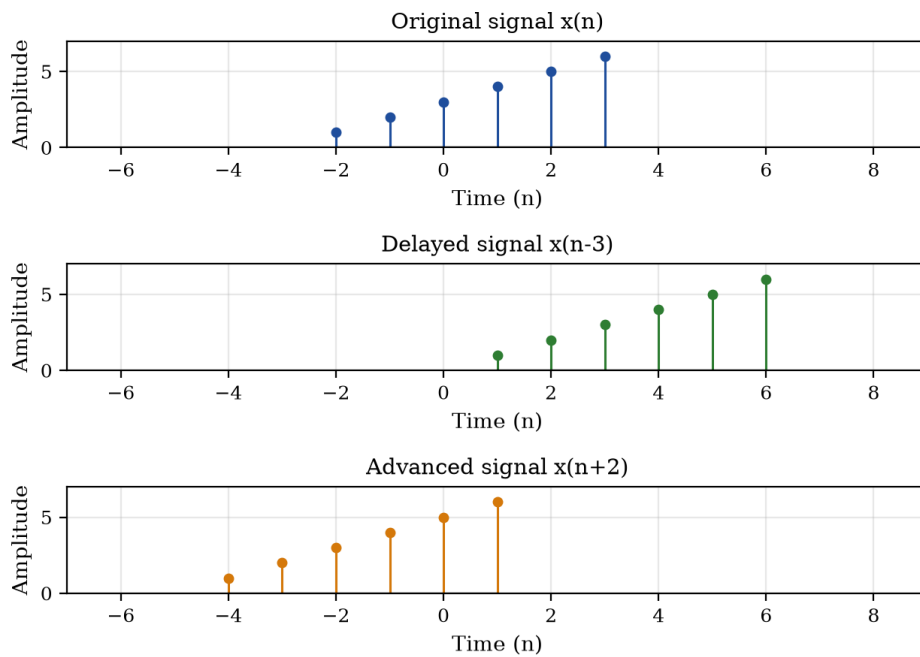


Fig. 2.2 Time shifting: delay by 3 samples and advance by 2 samples

3. Time Reversal

Time reversal (folding) reflects the sequence about the vertical axis $n = 0$. The sample at $n = m$ moves to $n = -m$, while $x(0)$ stays in place. In MATLAB *both* vectors must be reversed: the values with `flipr(x)` and the indices with `-flipr(n)`. Time reversal is a key step of convolution (Experiment 05).

$$y(n) = x(-n)$$

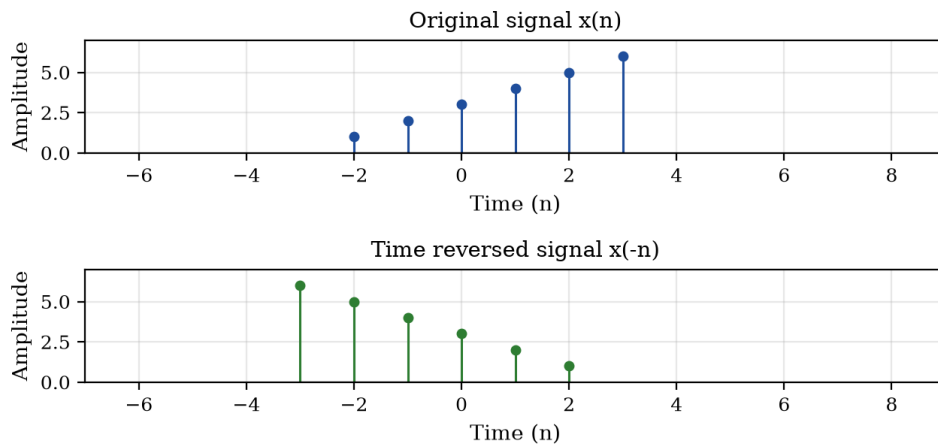


Fig. 2.3 Time reversal of $x(n)$

4. Time Scaling (Compression and Expansion)

For discrete-time signals the index must remain an integer, so time scaling behaves differently from the continuous-time case:

- **Compression (down-sampling) by an integer M:** $y(n) = x(Mn)$. Only every M-th sample is kept; the samples in between are *discarded* and cannot be recovered. For $M = 2$: $y(n) = \{x(-2), x(0), x(2)\} = \{1, 3, 5\}$ at $n = -1, 0, 1$.

- **Expansion (up-sampling) by an integer L:** $y(n) = x(n/L)$ when n is a multiple of L and 0 otherwise; $L - 1$ zeros are inserted between successive samples and no information is lost.

$$y(n) = \begin{cases} x(n/L), & n = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise} \end{cases}$$

These operations are the basis of sampling-rate conversion; the loss of samples in compression is closely related to aliasing (Experiment 04).

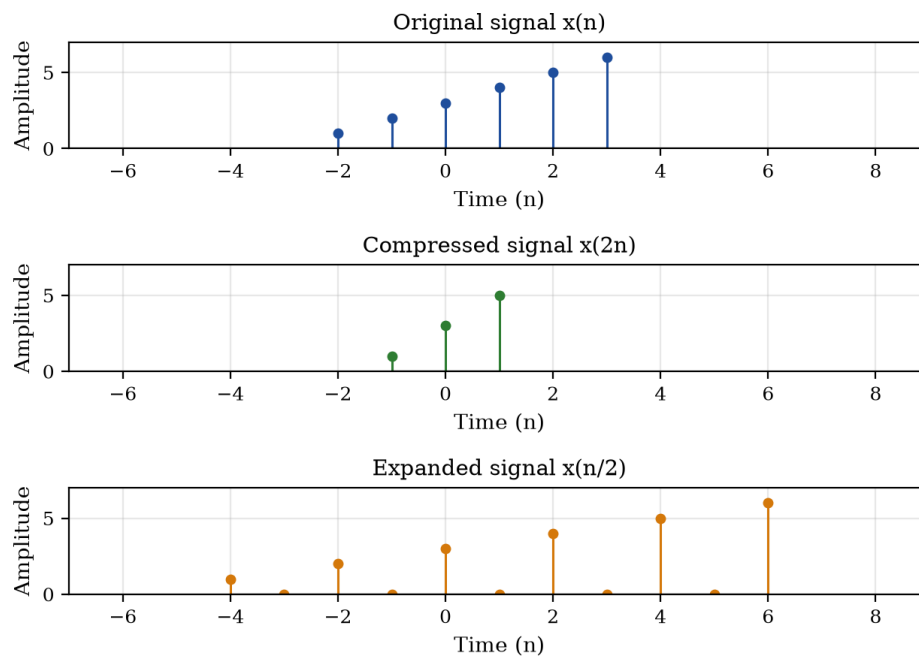


Fig. 2.4 Time compression by 2 and time expansion by 2

5. Combination of Operations

A general operation $y(n) = x(an + b)$ combines shifting with reversal or scaling. It is performed correctly by either (i) shifting first, $v(n) = x(n + b)$, and then scaling/reversing, $y(n) = v(an)$; or (ii) scaling/reversing first, $w(n) = x(an)$, and then shifting, $y(n) = w(n + b/a)$. For the common case

$$y(n) = x(-n + 2) = x(-(n - 2))$$

$x(n)$ is first reversed and the reversed signal is then *delayed* by 2 samples. The result is defined for $-1 \leq n \leq 4$, with $y(-1) = x(3) = 6$, $y(2) = x(0) = 3$ and $y(4) = x(-2) = 1$. Delaying first and reversing afterwards is a very common mistake: it gives $x(-n - 2)$ instead, as shown in Fig. 2.6.

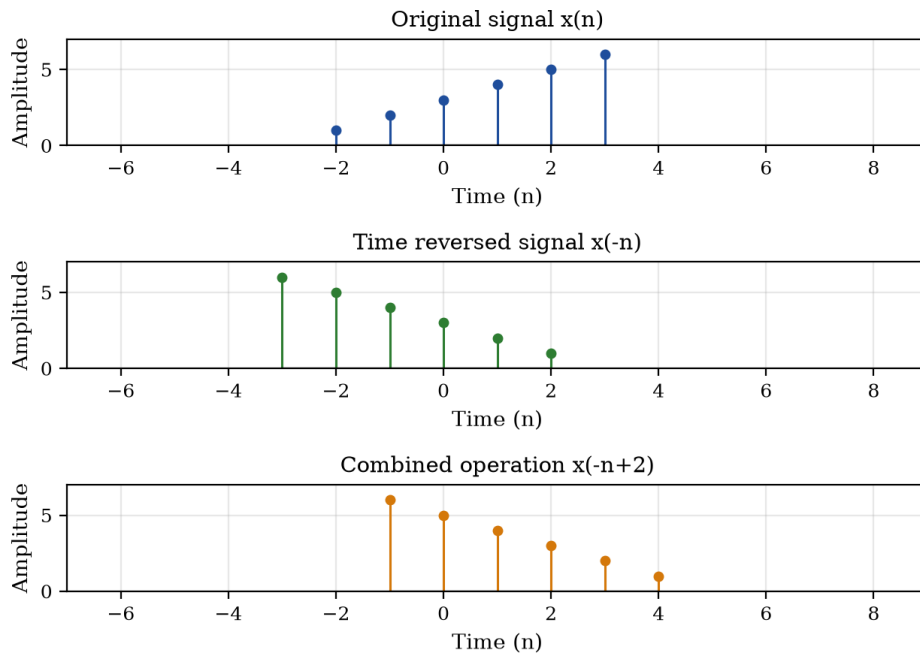


Fig. 2.5 Combined operation: reversal followed by a delay of 2 samples

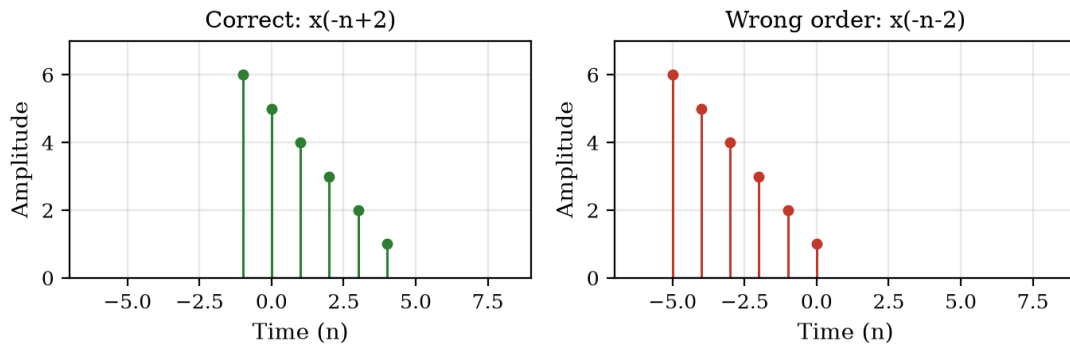


Fig. 2.6 Order of operations matters: correct $x(-n + 2)$ (left) and the wrong result $x(-n - 2)$ (right)

Important Equations

$$y(n) = A x(n) \quad (\text{amplitude scaling}) \quad (2.1)$$

$$y(n) = x(n - k), \quad k > 0 \quad (\text{delay}) \quad (2.2)$$

$$y(n) = x(n + k), \quad k > 0 \quad (\text{advance}) \quad (2.3)$$

$$y(n) = x(-n) \quad (\text{time reversal}) \quad (2.4)$$

$$y(n) = x(Mn) \quad (\text{compression by } M) \quad (2.5)$$

$$y(n) = x(n/L) \text{ for } n = mL, \quad 0 \text{ otherwise} \quad (\text{expansion by } L) \quad (2.6)$$

$$y(n) = x(-n + k) = x(-(n - k)) \quad (\text{reverse, then delay}) \quad (2.7)$$

Table 2.1 Commonly confused expressions

Expression	Operation	$x(0)$ moves to	MATLAB index rule
$x(n - k), k > 0$	Delay (right shift)	$n = k$	$n + k$
$x(n + k), k > 0$	Advance (left shift)	$n = -k$	$n - k$
$x(-n)$	Time reversal	$n = 0$	$\text{fliplr}(n)$, values $\text{fliplr}(x)$
$x(-n + k)$	Reverse, then delay by k	$n = k$	$\text{fliplr}(n) + k$
$x(-n - k)$	Reverse, then advance by k	$n = -k$	$\text{fliplr}(n) - k$
$x(Mn)$	Compression (samples lost)	$n = 0$	keep indices that are multiples of M
$x(n/L)$	Expansion (zeros inserted)	$n = 0$	$L \cdot \min(n) : L \cdot \max(n)$, fill every L -th point

MATLAB commands used

- **$n = a:b$** – creates the integer time-index vector $a, a+1, \dots, b$.
- **$A \cdot x$** – multiplies every element of x by the scalar A .
- **$\text{fliplr}(x)$** – reverses the order of the elements of a row vector.
- **ceil , floor** – round towards $+\infty$ and $-\infty$; used to find valid compressed indices.
- **$\text{zeros}(1, N)$** – creates a row vector of N zeros (used for expansion).
- **$\text{subplot}(m, p, k)$** – divides a figure into $m \times p$ panels and selects panel k .
- **$\text{stem}(n, x)$** – plots the discrete sequence x against the index n .
- **$\text{axis}([x_{\min} \ x_{\max} \ y_{\min} \ y_{\max}])$** – fixes the axis limits so that the panels can be compared.
- **$\text{disp}([n; \ x])$** – prints the index row and the value row in the command window.

PROCEDURE:**Apparatus Required**

- System with MATLAB 7.0 (no toolbox is required).

Algorithm

1. Define the time-index vector $n = -2 : 3$ and the sample vector $x = [1 \ 2 \ 3 \ 4 \ 5 \ 6]$.
2. Amplitude scaling: multiply x by the gains 2 and 0.5; keep the same index vector.
3. Time shifting: form the index of $x(n - 3)$ as $n + 3$ and the index of $x(n + 2)$ as $n - 2$; the values are unchanged.
4. Time reversal: reverse the values with $\text{fliplr}(x)$ and form the index $-\text{fliplr}(n)$.
5. Time scaling: for compression keep the samples whose index is a multiple of $M = 2$; for expansion create the index range $2 \cdot \min(n) \dots 2 \cdot \max(n)$, fill it with zeros and place x at every second position.
6. Combined operation: reverse $x(n)$ and then delay the reversed signal by 2 samples to obtain $x(-n + 2)$.

7. Plot the original and the resulting signals with stem() in subplots with the same axis limits.
8. Display the index and value vectors and compare them with the hand-calculated values.

Program

1. Amplitude Scaling

```
% 1. AMPLITUDE SCALING
clc;
clear all;
close all;
n = -2:3;                % time index
x = [1 2 3 4 5 6];       % x(-2)=1 ... x(3)=6
A1 = 2;                  % amplification factor
A2 = 0.5;                 % attenuation factor
y1 = A1*x;                % y(n) = 2x(n)
y2 = A2*x;                % y(n) = 0.5x(n)
subplot(3,1,1); stem(n,x); axis([-7 9 0 13]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Original signal x(n)');
subplot(3,1,2); stem(n,y1); axis([-7 9 0 13]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Amplitude scaled signal 2x(n)');
subplot(3,1,3); stem(n,y2); axis([-7 9 0 13]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Amplitude scaled signal 0.5x(n)');
```

2. Time Shifting

```
% 2. TIME SHIFTING
clc;
clear all;
close all;
n = -2:3;
x = [1 2 3 4 5 6];
k1 = 3;                  % delay in samples
nd = n + k1;             % index of x(n-3): right shift
k2 = 2;                  % advance in samples
na = n - k2;             % index of x(n+2): left shift
subplot(3,1,1); stem(n,x); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Original signal x(n)');
subplot(3,1,2); stem(nd,x); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Delayed signal x(n-3)');
subplot(3,1,3); stem(na,x); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Advanced signal x(n+2)');
disp('Delayed signal x(n-3):'); disp([nd; x]);
disp('Advanced signal x(n+2):'); disp([na; x]);
```

3. Time Reversal

```
% 3. TIME REVERSAL
clc;
clear all;
close all;
n = -2:3;
x = [1 2 3 4 5 6];
xr = fliplr(x);          % reverse the values
nr = -fliplr(n);         % index of x(-n)
subplot(2,1,1); stem(n,x); axis([-7 9 0 7]); grid on;
```

```

xlabel('Time (n)'); ylabel('Amplitude');
title('Original signal x(n)');
subplot(2,1,2); stem(nr,xr); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Time reversed signal x(-n)');
disp('Reversed signal x(-n):'); disp([nr; xr]);

```

4. Time Scaling (Compression and Expansion)

```

% 4. TIME SCALING (COMPRESSION AND EXPANSION)
clc;
clear all;
close all;
n = -2:3;
x = [1 2 3 4 5 6];
M = 2; % compression factor
nc = ceil(min(n)/M):floor(max(n)/M); % n for which M*n is valid
xc = x(M*nc - min(n) + 1); % y(n) = x(2n)
L = 2; % expansion factor
ne = L*min(n):L*max(n); % index of x(n/2)
xe = zeros(1,length(ne)); % start with all zeros
xe(1:L:length(ne)) = x; % x at every L-th point
subplot(3,1,1); stem(n,x); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Original signal x(n)');
subplot(3,1,2); stem(nc,xc); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Compressed signal x(2n)');
subplot(3,1,3); stem(ne,xe); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Expanded signal x(n/2)');
disp('Compressed signal x(2n):'); disp([nc; xc]);
disp('Expanded signal x(n/2):'); disp([ne; xe]);

```

5. Combined Operation $x(-n + 2)$

```

% 5. COMBINED OPERATION  $y(n) = x(-n+2)$ 
clc;
clear all;
close all;
n = -2:3;
x = [1 2 3 4 5 6];
xr = fliplr(x); % step 1: time reversal
nr = -fliplr(n);
ny = nr + 2; % step 2: delay the reversed signal by 2
y = xr;
subplot(3,1,1); stem(n,x); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Original signal x(n)');
subplot(3,1,2); stem(nr,xr); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Time reversed signal x(-n)');
subplot(3,1,3); stem(ny,y); axis([-7 9 0 7]); grid on;
xlabel('Time (n)'); ylabel('Amplitude');
title('Combined operation x(-n+2)');
disp('Combined signal x(-n+2):'); disp([ny; y]);

```

Program Explanation

MATLAB statement	What it does	Why it is used
clc; clear all; close all;	Clears the command window, deletes all workspace variables and closes all figures.	Old variables or plots from a previous program cannot affect the result.

MATLAB statement	What it does	Why it is used
<code>n = -2:3;</code>	Creates the row vector $[-2 \ -1 \ 0 \ 1 \ 2 \ 3]$ with the colon operator.	Stores the time indices; without it MATLAB cannot know where $n = 0$ lies.
<code>x = [1 2 3 4 5 6];</code>	Stores the six sample values; $x(1)$ is the value at $n = -2$ and $x(3)$ the value at $n = 0$.	An asymmetric sequence makes shifting and reversal clearly visible.
<code>y1 = A1*x; y2 = A2*x;</code>	Multiplies every sample by the gain 2 or 0.5.	Implements $y(n) = A x(n)$; the index n is reused because scaling does not move samples.
<code>nd = n + k1;</code>	Adds 3 to every index: $nd = [1 \ 2 \ 3 \ 4 \ 5 \ 6]$.	Implements the delay $x(n - 3)$: $x(0)$ moves to $n = 3$ (right shift).
<code>na = n - k2;</code>	Subtracts 2 from every index: $na = [-4 \ -3 \ -2 \ -1 \ 0 \ 1]$.	Implements the advance $x(n + 2)$: $x(0)$ moves to $n = -2$ (left shift).
<code>xr = fliplr(x);</code>	Reverses the order of the values: $[6 \ 5 \ 4 \ 3 \ 2 \ 1]$.	The last sample of x becomes the first sample of the folded signal.
<code>nr = -fliplr(n);</code>	Flips and negates the index vector: $[-3 \ -2 \ -1 \ 0 \ 1 \ 2]$.	Gives increasing indices for $x(-n)$; the value 6 (at $n = 3$) moves to $n = -3$.
<code>nc = ceil(min(n)/M):floor(max(n)/M);</code>	Finds the integers n for which $2n$ lies in $[-2, 3]$: $nc = [-1 \ 0 \ 1]$.	Only for these n is the argument $2n$ a valid index of x .
<code>xc = x(M*nc - min(n) + 1);</code>	Converts each time index $2n$ into a MATLAB position ($2n + 3$) and reads x there: $[1 \ 3 \ 5]$.	Translates signal time indices into MATLAB positions, which start at 1.
<code>ne = L*min(n):L*max(n);</code>	Builds the index range $-4, \dots, 6$ of the expanded signal.	$x(n/2)$ exists from $n = 2 \cdot (-2)$ to $n = 2 \cdot 3$.
<code>xe = zeros(1,length(ne));</code>	Creates a row of 11 zeros.	Positions where $n/2$ is not an integer must be zero.
<code>xe(1:L:length(ne)) = x;</code>	Writes the six samples into positions 1, 3, 5, ..., 11.	Places $x(m)$ at $n = 2m$ and leaves one zero between samples.
<code>ny = nr + 2; y = xr;</code>	Delays the reversed signal by 2: $ny = [-1 \ 0 \ 1 \ 2 \ 3 \ 4]$.	Implements $x(-n + 2) = x(-(n - 2))$ in the correct order.
<code>subplot(3,1,k); stem(n,x);</code>	Selects panel k of a 3×1 grid and draws the sequence as stems.	<code>stem()</code> is the correct plot for discrete-time sequences (Experiment 01).
<code>axis([-7 9 0 7]); grid on;</code>	Fixes the axis limits and adds a grid.	Identical axes in all panels make shifts and reversals directly comparable.
<code>xlabel(...); ylabel(...); title(...);</code>	Labels the axes and the panel.	Every plot in the lab record must be labelled.
<code>disp([nd; x]);</code>	Prints a 2-row matrix: time indices in row 1, values in row 2.	Allows the result to be compared with the hand calculation.

Precautions

- Do not confuse the MATLAB position with the time index: $x(1)$ is the first stored sample, not $x(0)$.
- For $x(n - k)$ add k to the index vector; for $x(n + k)$ subtract k . Check: where does $x(0)$ go?
- Time reversal needs both `flip1r(x)` and `-flip1r(n)`; reversing only one vector gives a wrong plot.
- $x(-n + k)$ is reverse-then-delay, *not* delay-then-reverse.
- Shift and scaling factors must be integers; $x(1.5)$ does not exist for a discrete-time signal.
- Use the same `axis([...])` in all subplots, otherwise a shift can look like no change at all.
- Use `stem()`, not `plot()`; `plot()` joins the samples and hides the zeros inserted by expansion.
- Do not use `length`, `max`, `min`, `sum` or `i` as variable names.

RESULTS:

Run each program in MATLAB and paste the resulting figure in the space provided below.

S.No.	Signal	Output Waveform (paste figure here)
1	Amplitude Scaling	
2	Time Shifting	
3	Time Reversal	
4	Time Scaling (Compression and Expansion)	

S.No.	Signal	Output Waveform (paste figure here)
5	Combined Operation $x(-n + 2)$	

Expected Output

- **Scaling:** the stems of $2x(n)$ are twice as tall (maximum 12) and those of $0.5x(n)$ half as tall (maximum 3); all stems stay at $n = -2 \dots 3$.
- **Delay / advance:** the whole pattern moves 3 samples to the right ($n = 1 \dots 6$) or 2 samples to the left ($n = -4 \dots 1$).
- **Reversal:** the sequence is mirrored about $n = 0$ ($n = -3 \dots 2$); $x(0) = 3$ is unchanged.
- **Compression:** only $\{1, 3, 5\}$ remain at $n = -1, 0, 1$; the values 2, 4 and 6 are lost.
- **Expansion:** eleven samples at $n = -4 \dots 6$ with a zero between every pair of original samples.
- **Combined $x(-n + 2)$:** the reversed sequence appears 2 samples to the right ($n = -1 \dots 4$); $y(2) = x(0) = 3$.

Table 2.2 Expected index ranges and sample values

Operation	Expected index range	Expected sample values	Observed (MATLAB)
$2x(n)$	$-2 \leq n \leq 3$	2, 4, 6, 8, 10, 12	
$0.5x(n)$	$-2 \leq n \leq 3$	0.5, 1, 1.5, 2, 2.5, 3	
$x(n - 3)$	$1 \leq n \leq 6$	1, 2, 3, 4, 5, 6	
$x(n + 2)$	$-4 \leq n \leq 1$	1, 2, 3, 4, 5, 6	
$x(-n)$	$-3 \leq n \leq 2$	6, 5, 4, 3, 2, 1	
$x(2n)$	$-1 \leq n \leq 1$	1, 3, 5	
$x(n/2)$	$-4 \leq n \leq 6$	1, 0, 2, 0, 3, 0, 4, 0, 5, 0, 6	
$x(-n + 2)$	$-1 \leq n \leq 4$	6, 5, 4, 3, 2, 1	

Observations

Conclusion

Thus the basic operations on discrete-time signals — amplitude scaling, time delay, time advance, time reversal, time compression, time expansion and the combined operation $x(-n + 2)$ — were performed using MATLAB, and the index ranges and sample values of the resulting signals were verified against the theoretical values.

Exercise:

1. For $x(n) = \{2, -1, 3, 0, 1\}$ with $n = -1, \dots, 3$, plot $x(n - 2)$, $x(n + 1)$, $x(-n)$ and $x(-n - 1)$ in one figure using subplot(4,1,k).
2. Change the compression factor to $M = 3$ and the expansion factor to $L = 3$. Record the new index ranges and sample values.
3. Generate $x(n) = 0.8^n u(n)$ for $0 \leq n \leq 10$ (Experiment 01) and plot $x(n)$, $x(n - 4)$ and $x(-n + 4)$.
4. Obtain $x(-n + 2)$ by the other correct order (advance by 2 first, then reverse) and verify that the result is identical.
5. Show by plotting that compressing by 2 and then expanding by 2 does *not* return the original signal, whereas expanding and then compressing does.
6. Compute $y(n) = 2x(-n + 1)$ for the test signal of this experiment by hand and verify it in MATLAB.

Viva Questions:

1. Why must a discrete-time signal be stored in MATLAB with a separate time-index vector?
2. Why does a delay $x(n - k)$ move the signal to the right even though k is subtracted?
3. Why is a delay physically realisable in real time while an advance is not?
4. What goes wrong if only the values are reversed with `fliplr(x)` but the index vector is left as n ?
5. Obtain $x(-n + 2)$ in two different correct orders. Which wrong order gives $x(-n - 2)$?
6. Why is information lost in compression $x(2n)$ but not in expansion $x(n/2)$?
7. In $x(n/2)$, why are the samples at odd n set to zero rather than interpolated?
8. What happens to the index range if the compression factor is changed to $M = 3$?
9. What is the effect of a negative amplitude scaling factor, e.g. $A = -1$?
10. Is time shifting a linear operation? Is it time-invariant?
11. Does time reversal change the energy of a finite sequence? Does compression?

12. Which operations of this experiment are used to compute a convolution sum?

Connection to the Next Experiment:

This experiment has shown that every operation on a discrete-time signal must track both the sample values and the time indices. Experiment 03 uses this skill directly: two signals can be added or multiplied sample by sample only after they are placed on a *common* time-index range. Experiment 03 then introduces the energy and power of a signal; the operations studied here provide useful checks — shifting and reversal do not change the energy of a finite sequence, while amplitude scaling by A multiplies it by A^2 .